

T3: Bidease Reporting API — портирование на PHP

Эталонная реализация: `bidease/bidease.py` (Python). Цель — портировать функциональность на PHP с сохранением структуры выходных данных. Документ самодостаточен: доступ к Python-коду не требуется. Версия документа: 2026-07-27.

1. Цель проекта

Разработать PHP-библиотеку для работы с Bidease Reporting API (mobile DSP). Библиотека загружает справочник кампаний и собирает дневную статистику (показы, клики, расход) на двух уровнях — кампании и креативы, — плюс строит сводную таблицу-агрегат `admin_audit`.

Выход каждой публичной функции — структурированный массив строк (эквивалент `pandas DataFrame` в Python-эталоне) с фиксированным набором и порядком колонок. Всего 4 публичные функции.

2. Общая логика работы библиотеки

API устроен принципиально проще большинства рекламных API: один синхронный GET-эндпоинт, отдающий CSV. Нет OAuth, нет `submit/poll/download`, нет пагинации, нет перебора сущностей.

- Группировка выполняется на сервере. Разрез задаётся повторяемым параметром `group` (например `group=Day&group=CampaignID`). Поэтому функциям 4.1–4.3 хватает одного HTTP-запроса на весь период, независимо от числа кампаний и дней. Исключение — `get_admin_audit` (4.4): она агрегирует чужие результаты и делает 2 запроса (статистика + справочник), а при пустой статистике — только 1.
- Справочников-эндпоинтов нет. Справочник кампаний собирается тем же `stats`-эндпоинтом: запрашиваются группировки-атрибуты за длинный период, метрики отбрасываются, строки дедуплицируются по `campaign_id`.
- Обогащение. К строкам добавляются константные и вычисляемые поля (раздел 3.7), затем колонки приводятся к фиксированному набору и порядку.

Не делать запрос на каждый день и не перебирать кампании — это лишняя нагрузка и другой результат (см. 3.6 про зависимость показов от разреза группировки).

3. Ключевые принципы работы с Bidease Reporting API

3.1. Авторизация

- Статический API-токен, который выдаёт техподдержка / CSM Bidease. Эндпоинтов выпуска или обновления токена не существует — перевыпуск только вручную через поддержку. TTL не документирован (токен долгоживущий).
- Токен передаётся `query`-параметром `api-token` в каждом запросе — не заголовком и не `Bearer`-токеном. Написание канонично через дефис.
- Хранить в переменной окружения `API_TOKEN`.
- К токenu привязана таймзона (по умолчанию UTC+0): все даты и часы В ДАННЫХ ОТВЕТА интерпретируются в ней.

Токен обязан быть замаскирован в ТЕКСТЕ исключений и в логах. HTTP-библиотеки кладут полный URL в текст ошибки (401 ... for url: ...?api-token=...), а раз токен едет в `query` — он утекает в логи вызывающего при любой ошибке. Python-эталон перевыбрасывает исключения с заменой значения на `<redacted>`, сохраняя тип исключения, код статуса и остальную диагностику. В PHP-порте сделать то же самое. ВАЖНО: маскируется именно текст — объекты запроса/ответа внутри исключения по-прежнему хранят сырой URL с токеном (`exc.response.url` в Python, `$e->getRequest()->getUri()` в Guzzle). Печатать/логировать URL из этих объектов НЕЛЬЗЯ — только санированный текст.

Границы периода Python-эталон вычисляет по ЛОКАЛЬНОЙ дате процесса (`date.today()`): это касается справочника (сегодня – 364 дня) и проверки лимита года. Если сервер приложения живёт не в таймзоне токена, возможен сдвиг границ на сутки. В PHP задавать таймзону явно и осознанно — либо повторить поведение эталона, либо согласовать с приёмником переход на UTC.

3.2. Базовый URL и используемый эндпоинт

BASE_URL = https://ui-api.bidease.com

Метод	Путь	Назначение
GET	/api/reporting/v1/stats	Единственный метод API: отчёт с произвольной группировкой

Все четыре функции библиотеки строятся на этом одном эндпоинте — разными комбинациями group и периода.

3.3. Схема получения данных

Метод синхронный: данные приходят сразу в теле ответа как text/csv. Пагинации нет — весь результат одним телом. UUID, polling и скачивание файлов отсутствуют как класс.

3.4. Лимиты API

Лимит	Значение	Как соблюдать
Диапазон дат	в пределах 1 года от текущей даты	Валидировать на входе (PERIOD_MAX_DAYS = 365)
Группировок в запросе	не более 7 значений group	Библиотека использует до 4 (справочник — 4, статистика — 2–3)
Rate limits	НЕ документированы	Защитный backoff при 429 (см. 3.5)
Пагинация	отсутствует	—

Про rate limits честно: в документации Bidease их нет, и на практике 429 ни разу не наблюдался. Область действия (на токен / на аккаунт), размер пула и период обнуления неизвестны. Поэтому backoff в клиенте — не реализация известного лимита, а защита на случай его появления. Если при промышленной нагрузке начнут приходить 429 — уточнять параметры у CSM Bidease.

3.5. Обработка ошибок и rate-limit

- HTTP 429 — повторить с экспоненциальным backoff: стартовая пауза 1 сек, удвоение, до 5 повторов. Уважать заголовок Retry-After, если он присутствует и задан числом секунд (форма HTTP-date не парсится — см. раздел 8).
- HTTP 401 — невалидный токен. Тело ответа ПУСТОЕ — сообщения об ошибке нет, диагностировать только по коду.
- HTTP 400 — тело plain-text: " -> fromdate should be less than todate".
- Тела ошибок — не CSV и не JSON, а короткий plain-text либо пустая строка. Парсер CSV к ним применять нельзя.

Тот же самый текст ошибки 400 приходит и когда fromdate выходит за лимит одного года — сообщение вводит в заблуждение. Поэтому период валидировать на стороне клиента ДО запроса, иначе причина ошибки будет неочевидна.

3.6. Подводные камни формата данных (CSV)

today — ЭКСКЛЮЗИВНАЯ граница. День today в отчёт НЕ входит. Это главная ловушка API. Публичные функции принимают date_from / date_to включительно, а в запрос уходит today = date_to + 1 день. Пример: данные за 21–22 июля → fromdate=2026-07-21&today=2026-07-23.

Формат значений day — MM/DD/YYYY 00:00:00 (американский порядок + время, НЕ ISO): 07/21/2026 00:00:00. Парсить явным форматом: НЕПУСТОЕ значение чужого формата обязано приводить к исключению, а не к молчаливому null — иначе битые даты тихо разъедутся по всей выгрузке. Уточнение по эталону: ПУСТОЕ значение day исключения не вызывает — строка доживает до результата с пустой датой. Отдельной фильтрации по дате нет, отбрасываются только строки без обязательных id.

- Структура заголовка CSV: сначала 35 колонок метрик, затем колонки группировок — в порядке их указания в запросе, именами в нижнем регистре (Day → day, CampaignID → campaignid, CreativeID → creativeid). Реальный заголовок богаче документации: в нём есть profit, roi, uniq_*, post_*, cr, ipm, roas, esra1-6, а заявленной в доках i2c — нет. Обращаться к колонкам по имени, не по индексу.
- Кодировка: тело — UTF-8, но заголовок Content-Type: text/csv идёт БЕЗ charset. В PHP тело приходит

байтами как есть — не применять к нему `utf8_encode / mb_convert_encoding` «на всякий случай», иначе кириллица в именах кампаний превратится в мусор. Разделитель — запятая, дробная часть чисел — точка.

- Числа приходят с высокой точностью: `spend = 17.24525999999991`. Приводить к `float` и округлять до 2 знаков (см. 3.7 и раздел 8).
- Битые и пустые числовые значения не роняют выгрузку и не теряют строку: `spend`, `impressions`, `clicks` проходят «привести к числу, иначе 0» — пустое значение или мусор (n/a) дают 0, а строка остаётся в результате. Не бросать исключение и не отбрасывать такую строку.
- Свежие даты (сегодня/вчера) могут «доезжать»: счётчики обновляются на лету. Для устоявшихся дат повторный запрос стабилен.

Показы зависят от РАЗРЕЗА группировки — это свойство API, а не баг. Один и тот же период, три разреза (факт, устоявшиеся данные):

Разрез	21.07	22.07
<code>group=Day</code>	187539	190533
<code>group=Day+CampaignID</code>	187327	190628
<code>group=Day+CampaignID+CreativeID</code>	186732	190676

Разброс 0.08–0.43 %. Клики и СЫРОЙ `spend` при этом совпадают во всех разрезах точно. Повторный запрос тем же разрезом даёт идентичные числа — дрейфа нет. Практический вывод: суммы показов между уровнями (аудит ↔ кампании ↔ креативы) сходиться «до единицы» не обязаны; сверять по `clicks`.

Отдельно про расход: сырой `spend` совпадает, но выгружаемый `costs_usd` округляется ПОСТРОЧНО, а строк на разных уровнях разное количество — поэтому суммы по дню могут разойтись на копейки. Факт на данных этого ТЗ: за 22.07 кампании дают 571.01, креативы — 571.00. Сверку уровней по расходу вести с допуском ± 0.01 на день, а не на точное равенство.

Поведение при пустом результате: запрос С группировками → только строка заголовка, 0 строк данных; запрос БЕЗ группировок → одна строка нулей по всем метрикам. Второе не путать с реальными нулями. Пустой ответ — не ошибка: функция должна вернуть пустой результат с правильным набором колонок.

3.7. Обогащение строк (соглашение проекта)

Константные и вычисляемые поля добавляются на стороне клиента ДО финальной сборки колонок.

Значения констант — примеры, заменяются при интеграции.

- Для всех функций: `account_id = 1`, `source_type_id = 10`.
- Справочник кампаний: `product_name = "prod_test"`, `camp_type = "camp_test"`, `camp_category = "cat_test"`, `owner_id = 1`. Внимание: `product_id` — РЕАЛЬНЫЙ `ProductID` из API (не константа, в отличие от других проектов-источников).
- Расход — ровно одно поле `costs_usd` ← API-поле `spend` как есть, округление до 2 знаков.
- Составные ключи: `id_key_camp = "1_" + campaign_id` (все функции); `id_key_ad = id_key_camp + "_" + creative_id` (только уровень креативов). Префикс "1_" в эталоне ЗАХАРДКОЖЕН и не пересчитывается из `account_id`; если при интеграции меняется `account_id`, решение о префиксе принимается отдельно.

Валюта — доллары США. Videase отдаёт расход в USD, поэтому рублёвые поля конвенции (`costs_nds`, `costs_without_nds`, `costs_nds_ak`, `costs_without_nds_ak`) и константа агентской комиссии `ak` в этом источнике НЕ применяются и в выгрузке отсутствуют. НДС не считается, курс не применяется — пересчёт валюты и налогов выполняет приёмник данных.

Ключ `id_key_ad` ДВУХЗВЕННЫЙ: в Videase нет уровня «группа объявлений», иерархия — кампания → креатив.

4. Публичные функции

Колонки и их порядок — фиксированные контракты.

Все stat-функции принимают `date_from / date_to` (YYYY-MM-DD) ВКЛЮЧИТЕЛЬНО. Валидация до запроса — ровно три проверки, как в эталоне: (1) обе даты парсятся строго как YYYY-MM-DD, иначе исключение; (2) `date_from <= date_to`; (3) `date_from` не старше 365 дней от текущей даты (ровно -365 проходит, -366 нет). При нарушении — исключение ДО обращения к API. Даты разбираются по формату; канонический вид — YYYY-MM-DD, но эталонный парсер принимает и непаддированный YYYY-M-D (2026-7-1), а 21.07.2026 и 2026/07/01 отвергает (в PHP `strtotime` проглотит и их — нужен явный разбор по формату; на выходе `date` всегда YYYY-MM-DD). Чего эталон НЕ проверяет (и порт не должен добавлять от себя): длину интервала, `date_to` в будущем, существование данных за период.

Порядок строк в результате. Функции 4.1–4.3 сохраняют порядок строк как их отдал API (без сортировки — CSV приходит неупорядоченным по дате). `get_admin_audit` (4.4) отсортирован по ключам группировки (`date`, затем `account_id`, `source_type_id`, `owner_id`) — следствие группировки. Если приёмник сравнивает выгрузки построчно, сортировать нужно на его стороне.

4.1. `get_campaign_dict()` — справочник кампаний

Запрос: GET `/api/reporting/v1/stats c`

`group=CampaignID&group=CampaignName&group=AdvertiserID&group=ProductID`, период — последний год (`fromdate` = сегодня - 364 дня, `todate` = сегодня). Метрики из ответа отбрасываются; строки дедуплицируются по `campaign_id` (выживает ПЕРВАЯ В ПОРЯДКЕ ОТВЕТА API — не «свежайшая» и не по сортировке), строки без `campaign_id` отбрасываются.

В справочник попадают только кампании, имевшие хотя бы одно событие за период — следствие того, что справочник собирается из отчёта. Кампании без открутки не видны.

Здесь `todate` НЕ увеличивается на день — в отличие от stat-функций (раздел 5, шаг 2) эталон передаёт `todate` = сегодня как есть. Поскольку граница эксклюзивна, окно справочника фактически ЗАКАНЧИВАЕТСЯ ВЧЕРА: кампания, начавшая откручиваться сегодня, попадёт в справочник только завтра. Это осознанное поведение эталона — воспроизвести как есть, не «чинить» добавлением дня.

Поле	Тип	Источник
<code>campaign_id</code>	integer	CSV <code>campaignid</code> (единственное поле с явным приведением к целому)
<code>campaign_name</code>	string	CSV <code>campaignname</code>
<code>advertiser_id</code>	integer или NULL	CSV <code>advertiserid</code> — приведения в эталоне НЕТ, см. предупреждение ниже
<code>account_id</code>	integer	константа 1
<code>source_type_id</code>	integer	константа 10
<code>product_id</code>	integer или NULL	CSV <code>productid</code> — РЕАЛЬНЫЙ ID продукта; та же оговорка, что и у <code>advertiser_id</code>
<code>product_name</code>	string	константа "prod_test"
<code>camp_type</code>	string	константа "camp_test"
<code>camp_category</code>	string	константа "cat_test"
<code>id_key_camp</code>	string	вычисляется: "1_" + <code>campaign_id</code>
<code>owner_id</code>	integer	константа 1

`advertiser_id` и `product_id` НЕ приводятся к целому и могут быть NULL. Явное приведение в эталоне есть только у `campaign_id`. Если в ответе API эти поля пусты, строка ОСТАЁТСЯ в справочнике, а значения уходят в выгрузку как NULL (в Python тип колонки становится дробным — 29153.0). PHP-порт не должен кастовать их к int: (int) null даст 0, и в приёмник уедет ложный `product_id = 0` вместо «значение неизвестно».

4.2. `get_campaigns_daily_stat(date_from, date_to)` — статистика по кампаниям

Запрос: `group=Day&group=CampaignID`, один GET на весь период. Гранулярность: одна строка на день × кампанию (уникальность обеспечивает серверная группировка — дедупликация на клиенте не нужна). Строки без `campaign_id` отбрасываются.

Поле	Тип	Источник
date	string	CSV day (MM/DD/YYYY 00:00:00)
	YYYY-MM-DD	
campaign_id	integer	CSV campaignid
impressions	integer	CSV impressions
clicks	integer	CSV clicks
costs_usd	float	CSV spend, округление до 2 знаков (доллары США)
account_id	integer	константа 1
source_type_id	integer	константа 10
id_key_camp	string	вычисляется: "1_" + campaign_id

Названия кампаний в статистике HET — джойнить из справочника по campaign_id (либо добавить group=CampaignName, но тогда разрез изменится, см. 3.6).

4.3. get_creatives_daily_stat(date_from, date_to) — статистика по креативам

Запрос: group=Day&group=CampaignID&group=CreativeID, один GET на весь период. Гранулярность: одна строка на день × кампанию × креатив. Строки без campaign_id ИЛИ без creative_id отбрасываются (оба нужны для ключа).

Поле	Тип	Источник
date	string	CSV day
campaign_id	integer	CSV campaignid
creative_id	integer	CSV creativeid
impressions	integer	CSV impressions
clicks	integer	CSV clicks
costs_usd	float	CSV spend, round(2)
account_id	integer	константа 1
source_type_id	integer	константа 10
id_key_camp	string	"1_" + campaign_id
id_key_ad	string	id_key_camp + "_" + creative_id

Один креатив может крутиться в нескольких кампаниях — строки при этом разделены по campaign_id, и id_key_ad у них различается. Это корректно.

4.4. get_admin_audit(date_from, date_to) — сводный аудит

Собственного запроса к API нет — это агрегат поверх 4.2:

- Взять результат get_campaigns_daily_stat(date_from, date_to); пустой → вернуть пустой результат, справочник НЕ запрашивать.
- Взять owner_id из get_campaign_dict(), приджойнить по campaign_id.
- Сгруппировать по date × account_id × source_type_id × owner_id, просуммировать impressions, clicks, costs_usd; сумму costs_usd округлить до 2 знаков.
- Добавить chef_flag = 1. Итого 2 HTTP-запроса на вызов.

Кампания может быть в статистике, но отсутствовать в справочнике (период справочника — 364 дня). Тогда owner_id пустой → подставить 1 ДО группировки, иначе строка молча выпадет из агрегата.

Поле	Тип	Источник
date	string	ключ группировки
account_id	integer	константа 1 (ключ группировки)
source_type_id	integer	константа 10 (ключ группировки)
owner_id	integer	join из справочника (пусто → 1; ключ группировки)
impressions	integer	сумма по дню
clicks	integer	сумма по дню
costs_usd	float	сумма по дню, round(2)
chef_flag	integer	константа 1

5. Алгоритм сбора статистики (общий шаблон)

1. Валидировать период: `date_from <= date_to`; `date_from` не старше 365 дней от сегодня. Нарушение -> исключение ДО HTTP-запроса.
2. Собрать query: `api-token`, `fromdate=date_from`, `todate=date_to + 1 день`, затем повторяемые `group=...` (порядок группировок определяет порядок колонок-группировок в CSV).
ИСКЛЮЧЕНИЕ: справочник (4.1) передаёт `todate=сегодня` БЕЗ прибавления дня.
3. GET `/api/reporting/v1/stats` -> `text/csv`.
429 -> экспоненциальный backoff (1, 2, 4, 8, 16 сек; максимум 5 повторов).
4. Распарсить CSV по ИМЕНАМ колонок заголовка. Пустое тело или только заголовок -> пустой результат с правильным набором колонок (не ошибка).
5. Переименовать колонки-группировки:
`day->date`, `campaignid->campaign_id`, `campaignname->campaign_name`,
`advertiserid->advertiser_id`, `productid->product_id`,
`creativeid->creative_id`.
6. Привести типы:
 - `date`: `MM/DD/YYYY HH:MM:SS` -> `YYYY-MM-DD` явным форматом (пустое -> `NULL`, чужой непустой формат -> исключение);
 - обязательные `id` (`campaign_id`, `creative_id`) -> целое; пустое значение и строки-сентинелы из списка раздела 8 (`NA`, `N/A`, `n/a`, `NULL`, `null`, `NaN`, `None`, `<NA>`, `#N/A` и т.д.) считаются отсутствующими: строка отбрасывается на шаге 7. Любая другая нечисловая строка – включая прочерк "-" – роняет процесс исключением (так ведёт себя эталон);
 - `impressions`, `clicks`, `spend`: "привести к числу, иначе 0" – пустое и мусорное значение дают 0, строка НЕ теряется; `costs_usd = round(spend, 2)`;
 - `advertiser_id`, `product_id` (справочник): НЕ приводить, сохранять `NULL`.
7. Отбросить строки без обязательных `id` (см. описание функции).
Проверять именно на `NULL`: `id` со значением 0 – валиден.
8. Обогащать: константы, составные ключи (раздел 3.7).
9. Привести к фиксированному набору и порядку колонок.

6. Примеры запросов и ответов API (реальные)

6.1. Форма запроса

```
GET https://ui-api.bidease.com/api/reporting/v1/stats
?api-token={API_TOKEN}
&fromdate=2026-07-21
&todate=2026-07-23      <- эксклюзивно: данные по 22.07 включительно
&group=Day
&group=CampaignID
```

Ответ: HTTP 200, Content-Type: text/csv (без charset!)

6.2. Заголовок CSV (реальный, живой API)

Одна строка; здесь разбита для читаемости. 35 метрик, затем колонки группировок:

```
conversions,spend,profit,roi,impressions,uniq_impressions,clicks,
uniq_clicks,post_clicks,post_views,ctr,ecpm,ecpc,cr,ipm,cpi,roas,ad_roas,
ad_purchases,ad_roas_purchases,revenue,iap_sum,iap,goal1,goal2,goal3,goal4,
goal5,goal6,есра1,есра2,есра3,есра4,есра5,есра6,day,campaignid
```

Библиотеке нужны только `spend`, `impressions`, `clicks` и колонки группировок. Остальные метрики (включая расчётные `ctr`, `ecpm`, `ecpc`, `cpi` и `conversions / revenue / iap* / goal1-6`) в выгрузку не берутся — решение проекта.

6.3. Статистика по кампаниям (group=Day&group=CampaignID)

Строка данных целиком, как её отдаёт API:

```
2,17.24525999999991,0,0,5788,2766,406,0,0,0,0.07014512785072564,
2.9794851416724097,0.04247600985221653,0,0.346,8.622629999999955,0,0,0,0,
17.24525999999991,0,0,2,1,0,0,0,8.622629999999955,17.24525999999991,0,0,0,0,
```


07/22/2026 00:00:00,154402

Читается так: spend = 17.24525999999991 → costs_usd = 17.25; impressions = 5788; clicks = 406; day = 07/22/2026 00:00:00 → date = 2026-07-22; campaignid = 154402.

6.4. Справочник (group=CampaignID+CampaignName+AdvertiserID+ProductID)

Хвост строки — колонки группировок в порядке запроса:

...,154369,x5_x5_igronik_Пятерочка_МояВыгода_android,2608,29153

...,154402,x5_x5_igronik_Пятерочка_МояВыгода_ios,2608,29154

campaignid=154369, campaignname=..._android (кириллица, UTF-8), advertiserid=2608, productid=29153.

6.5. Креативы (group=Day+CampaignID+CreativeID)

...,07/22/2026 00:00:00,154369,647446

...,07/21/2026 00:00:00,154402,647448

6.6. Ошибки (реальные ответы)

HTTP 400, тело (plain-text): " -> fromdate should be less than todate"

Причина 1: fromdate >= todate

Причина 2: fromdate старше 1 года <- ТОТ ЖЕ текст (вводит в заблуждение)

HTTP 401, тело: "" (пусто)

Причина: невалидный api-token

7. Примеры таблиц на выходе

Реальные строки, период 21–22.07.2026.

7.1. get_campaign_dict — справочник кампаний

campaign_id	campaign_name	advertiser_id	product_id
154369	x5_x5_igronik_Пятерочка_МояВыгода_android	2608	29153
154402	x5_x5_igronik_Пятерочка_МояВыгода_ios	2608	29154

Константные и вычисляемые поля тех же двух строк:

account_id	source_type_id	product_name	camp_type	camp_category	id_key_camp	owner_id
1	10	prod_test	camp_test	cat_test	1_154369	1
1	10	prod_test	camp_test	cat_test	1_154402	1

7.2. get_campaigns_daily_stat — статистика по кампаниям

date	campaign_id	impressions	clicks	costs_usd	account_id	source_type_id	id_key_camp
2026-07-21	154369	179576	7063	531.86	1	10	1_154369
2026-07-21	154402	7751	519	23.09	1	10	1_154402
2026-07-22	154369	184840	7006	553.76	1	10	1_154369
2026-07-22	154402	5788	406	17.25	1	10	1_154402

7.3. get_creatives_daily_stat — статистика по креативам

Полная выгрузка за период — все 8 строк, В ТОМ ПОРЯДКЕ, В КАКОМ ИХ ОТДАЛ API. (Таблицы 7.1 и 7.2 отсортированы здесь для читаемости — эталон их не сортирует; а вот 7.4 отсортирована по-настоящему: агрегат группируется по ключам, см. раздел 4.) Колонки account_id = 1 и source_type_id = 10 опущены для компактности.

date	campaign_id	creative_id	impressions	clicks	costs_usd	id_key_ad
2026-07-22	154369	647446	20743	1157	61.81	1_154369_647446
2026-07-21	154402	647448	1415	118	4.21	1_154402_647448
2026-07-21	154369	647446	17085	1031	50.91	1_154369_647446
2026-07-21	154369	647445	161895	6032	480.95	1_154369_647445
2026-07-21	154402	647447	6337	401	18.88	1_154402_647447
2026-07-22	154402	647448	1134	113	3.38	1_154402_647448
2026-07-22	154369	647445	164144	5849	491.94	1_154369_647445

2026-07-22	154402	647447	4655	293	13.87	1_154402_647447
------------	--------	--------	------	-----	-------	-----------------

7.4. get_admin_audit — сводный аудит

date	account_id	source_type_id	owner_id	impressions	clicks	costs_usd	chef_flag
2026-07-21	1	10	1	187327	7582	554.95	1
2026-07-22	1	10	1	190628	7412	571.01	1

Проверка сходимости: аудит за 21.07 = 179576 + 7751 = 187327 показов и 531.86 + 23.09 = 554.95 USD — совпадает с 7.2 точно (аудит и считается из 7.2). А с креативами (7.3) сходимость НЕ точная, и это нормально: показы за 21.07 по креативам = 186732 против 187327 у кампаний (−595) — свойство API из 3.6; расход за 22.07 по креативам = 61.81 + 3.38 + 491.94 + 13.87 = 571.00 против 571.01 у кампаний — следствие построчного округления на разной гранулярности. Сверять уровни: показы — без ожидания точного равенства, расход — с допуском ±0.01 на день, клики — точно.

8. Рекомендации по реализации на PHP

- Версия: PHP >= 8.1, Composer-проект (типизированные свойства, enum, readonly).
- HTTP-клиент: Guzzle (или Symfony HttpClient) с retry-middleware на 429; таймаут запроса — 30 сек.
- Повторяемые query-параметры — частая точка отказа: group передаётся несколько раз, а `http_build_query(['group' => ['Day','CampaignID']])` даст `group%5B0%5D=Day...`, которое API не поймёт; массив с одинаковыми ключами PHP просто схлопнет. Собирать строку явно (пример ниже) и проверить глазами итоговый URL: должно быть ровно `group=Day&group=CampaignID`.
- Парсинг CSV: `league/csv` либо `fgetcsv` по потоку — полноценный RFC 4180-парсер, НЕ `explode(',')`: названия кампаний задаёт клиент, и запятая или кавычка внутри имени разъедет всю строку. Читать по именам колонок из первой строки, не по индексам (реальный заголовок шире документации и может расширяться).
- Строки-сентинелы: Python-парсер сам превращает часть строк в NULL, а в PHP они останутся обычными строками. Разница: `campaign_id = "NA"` в эталоне делает строку «без id» (она отбрасывается), а наивный `(int)"NA"` в PHP даст 0 — и строка уедет в выгрузку с ложным id. Полный список (19 значений, РЕГИСТР ВАЖЕН): пустая строка, NA, N/A, n/a, #NA, #N/A, #N/A N/A, NULL, null, NaN, nan, -NaN, -nan, <NA>, None, 1.#IND, 1.#QNAN, -1.#IND, -1.#QNAN. Прочерк "-" в список НЕ входит (как и NONE, Null): для эталона это обычная строка, и на `campaign_id = "-"` он ПАДАЕТ с ошибкой приведения к целому, а не отбрасывает строку. Не «додумывать» лишние сентинелы.
- Falsy-ловушка PHP: `id = 0` — валидное значение, а `empty($id) / if (!$id)` его отбросят. Строки фильтровать строго по «значение отсутствует» (null), а не по ложности.
- Кодировка: тело — UTF-8; никаких перекодировок не применять (см. 3.6).
- Даты: `DateTimeImmutable::createFromFormat('m/d/Y H:i:s', $v)` — и обязательно проверять результат на false, а также `DateTime::getLastErrors()`: молчаливо принятая битая дата хуже исключения. Эксклюзивный `today`: `(new DateTimeImmutable($dateTo))->modify('+1 day')->format('Y-m-d')`.
- Пустые ответы: пустое тело и «только заголовок» — штатный результат; возвращать пустой массив с корректным набором колонок, не бросать исключение.
- Контракт обработки ошибок (повторить поведение эталона): ретраится ТОЛЬКО HTTP 429; 400, 401, 5xx, таймауты и сетевые сбои пробрасываются вызывающему как есть — без «проглатывания», без подстановки пустого результата и без повторов. Retry-middleware в Guzzle по умолчанию ретраит и 5xx, и сетевые ошибки — это ДРУГОЕ поведение, ограничить его строго 429. Заголовок `Retry-After` эталон читает как число секунд; форма `HTTP-date` не парсится — в этом случае клиент откатывается к собственному `backoff` и НЕ падает. Повторить это поведение: разбор заголовка не должен ронять выгрузку. Единственная обработка ошибки — маскировка токена в тексте исключения (см. 3.1).
- Формат возврата: эталон отдаёт таблицу (pandas DataFrame). В PHP разумный эквивалент — массив ассоциативных массивов с ключами-именами колонок в заданном порядке; «нет значения» передавать как null (не 0 и не ""), иначе NULL-семантика справочника (см. 4.1) потеряется.
- Логирование: PSR-3. Пропуски строк (без `campaign_id / creative_id`) — рекомендуемый уровень warning; в Python-эталоне они отбрасываются молча, так что это улучшение порта, а не воспроизведение поведения. При логировании URL маскировать `api-token`.

Сборка повторяемых query-параметров (рабочий пример):

```
$pairs = [
    ['api-token', $token],
    ['fromdate', $dateFrom],
    ['todate', $todateExclusive],
    ['group', 'Day'],
    ['group', 'CampaignID'],
];
$qqs = implode('&', array_map(
    static fn(array $p): string => rawurlencode($p[0]) . '=' . rawurlencode($p[1]),
    $pairs
));
$response = $client->request('GET', self::STATS_PATH . '?' . $qqs);
```

Деньги — главное расхождение платформ, читать внимательно. Python-эталон умножает значение на 100, округляет half-to-even по ДВОИЧНОМУ double и делит обратно. Встроенный PHP round() использовать НЕЛЬЗЯ ни в каком режиме — ни round(\$v, 2), ни round(\$v, 2, PHP_ROUND_HALF_EVEN), ни масштабирование round(\$v * 100, 0, PHP_ROUND_HALF_EVEN) / 100: PHP округляет от ДЕСЯТИЧНОГО представления и применяет pre-rounding по значащим цифрам (в 8.1–8.3; в 8.4 round() переработан), то есть результат зависит ещё и от версии интерпретатора. На равномерной выборке значений вида x.xx5 «десятичный half-even» расходится с эталоном в 5.7 % случаев (573 из 10 000). Эталон воспроизводит только явный half-even по двоичному double — без обращения к round().

```
/** Округление до 2 знаков ровно как в Python-эталоне (numpy rint, ничья к чётному). */
function roundHalfEven2(float $v): float
{
    $scaled = $v * 100.0;
    $floor = floor($scaled);
    $diff = $scaled - $floor;

    if ($diff > 0.5) {
        $r = $floor + 1.0;
    } elseif ($diff < 0.5) {
        $r = $floor;
    } else {
        // ровно .5 - к чётному
        $r = (fmod($floor, 2.0) == 0.0) ? $floor : $floor + 1.0;
    }

    return $r / 100.0;
}

$costsUsd = roundHalfEven2($spend);
```

Алгоритм сверен с эталоном на 220 000 КОНЕЧНЫХ значений (без INF/NaN и без переполнения при умножении на 100) — случайные расходы диапазона Videase, все «половинки» вида x.xx5, отрицательные, большие (до 1e12) и краевые (0, ±1e-9, 0.005, 1e15): расхождений ноль. Единственное известное отличие — знак нуля: на точной ничьей около -0.005 эталон может дать -0.0, а этот код вернёт 0.0; для денег и колонки numeric(18,2) это неразлично. Проверка выполнена на стороне эталона (Python), живого PHP под рукой не было — поэтому при приёмке прогнать таблицу векторов ниже на целевой версии PHP. Если какой-то вектор разойдётся, причина будет в float-семантике конкретной сборки, а не в алгоритме: он намеренно не использует round().

Тест-векторы для приёмки (проверены на pandas 2.2.3 и 3.0.2 — значения идентичны):

Вход	Ожидаемый costs_usd	Вход	Ожидаемый costs_usd
12.345	12.34	0.285	0.28
12.346	12.35	0.125	0.12
2.675	2.68	0.545	0.55
1.005	1.00	0.575	0.57
556.31534	556.32		

Смягчающее обстоятельство: реальные spend приходят с большим числом знаков (17.24525999999991) и в точные «половинки» практически не попадают — риск сосредоточен в фикстурах тестов, по которым и

будут сверять порт. Для внутреннего учёта деньги хранить в decimal-типе БД (numeric(18,2)), не во float. Константы (не хардкодить в коде вызовов):

Константа	Значение
HTTP_TIMEOUT_SEC	30
PERIOD_MAX_DAYS	365
MAX_GROUPS	7
RATE_LIMIT_BASE_SEC	1
RATE_LIMIT_RETRY_MAX	5
DICT_PERIOD_DAYS	364 (период справочника)

Тесты: моки HTTP на все 4 функции; пустое тело и «только заголовок»; строки без campaign_id / creative_id; невалидный период (исключение ДО запроса); битый и пустой spend → costs_usd = 0.0 без потери строки; повтор при 429.

9. Критерии приёмки

- [] Все 4 публичные функции возвращают данные с теми же колонками и в том же порядке, что в Python-эталоне bidease.py (разделы 4.1–4.4).
- [] todate передается эксклюзивно: вызов за 21–22.07 уходит в API как fromdate=2026-07-21&today=2026-07-23 и возвращает ровно два дня.
- [] date — строки YYYY-MM-DD; исходный MM/DD/YYYY 00:00:00 распарсен явным форматом. Непустое значение чужого формата даёт исключение; пустое значение — NULL, и строка остаётся в выгрузке.
- [] Расход отдаётся ЕДИНСТВЕННЫМ полем costs_usd (доллары США, 2 знака); полей costs_nds / costs_without_nds / costs *_ak / ak в выгрузке НЕТ.
- [] id_key_camp = "1_" + campaign_id; id_key_ad = id_key_camp + "_" + creative_id (двухзвенный, без уровня группы).
- [] get_admin_audit: суммы clicks и costs_usd совпадают с суммами get_campaigns_daily_stat по тому же дню; кампания, отсутствующая в справочнике, не теряется (owner_id = 1).
- [] Пустой период → пустой результат с правильными колонками, без исключений.
- [] Невалидный период (date_to < date_from; выход за 365 дней) → исключение ДО HTTP-запроса.
- [] Кириллица в campaign_name не искажена (проверять на реальном ответе).
- [] Токен не попадает в текст исключений и в логи: вызов с заведомо неверным токеном даёт исключение, где вместо значения стоит api-token=<redacted>, а код статуса и остальная диагностика сохранены. URL из объекта запроса/ответа нигде не логируется (там токен остаётся сырым).
- [] Битый или пустой spend / impressions / clicks → 0, строка не теряется и исключения нет.
- [] Округление денег совпадает с таблицей тест-векторов раздела 8 (в частности 12.345 → 12.34, 0.545 → 0.55, 0.575 → 0.57).
- [] advertiser_id / product_id без значения приходят как NULL, а не 0.
- [] Строки с campaign_id = 0 / creative_id = 0 НЕ отбрасываются.
- [] Порядок строк функций 4.1–4.3 — как отдал API (без сортировки); get_admin_audit отсортирован по ключам группировки.
- [] Справочник запрашивается с today = сегодня без прибавления дня (окно заканчивается вчера).
- [] Ретраится только 429; 400, 401, 5xx, таймауты и сетевые сбои пробрасываются.
- [] Число HTTP-запросов: по одному на вызов функций 4.1–4.3; get_admin_audit — два (и один, если статистика пуста).
- [] README с примерами вызова + .env.example (API_TOKEN).